COSC2633/2637 Big Data Processing

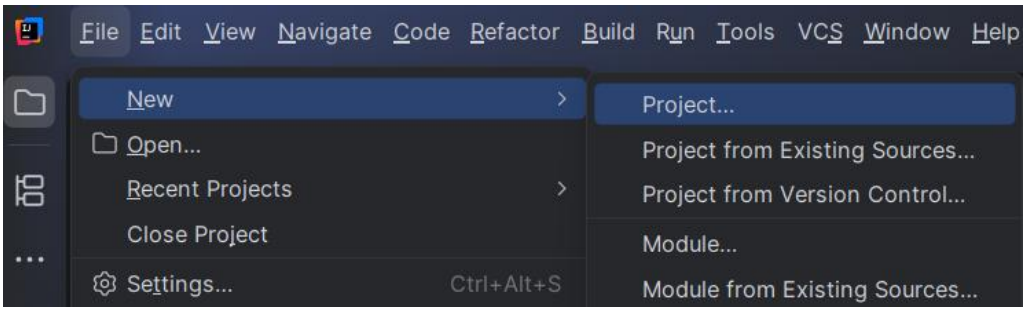
Week 10 - Lab Task

Scala Maven Project on Intellij IDEA

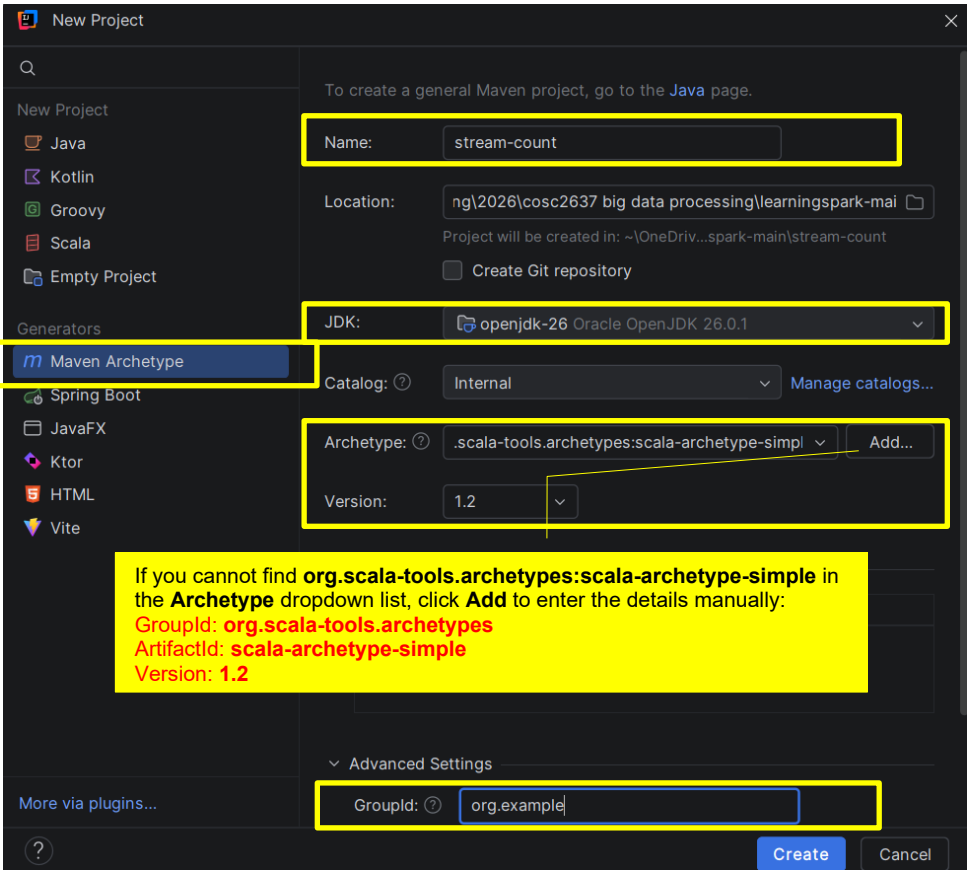
Install IntelliJ IDEA and install the required plugin **scala** into our IntelliJ; after installation, you need restart Intellij IDEA (details can be found in Week 9 – Lab Task).

Task: create an executable Jar for monitoring a socket on localhost

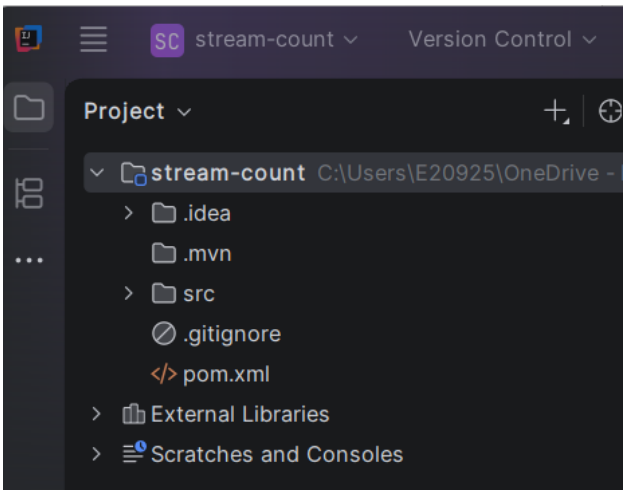
- Create Scala Maven Project



- In **New Project** window, select **Maven Archetype,** name the project such as **stream-count,** select/text **org.scala-tools.archetypes:scala-archetype-simple** for **Archetype**, name **GroupId** such as **org.example**.

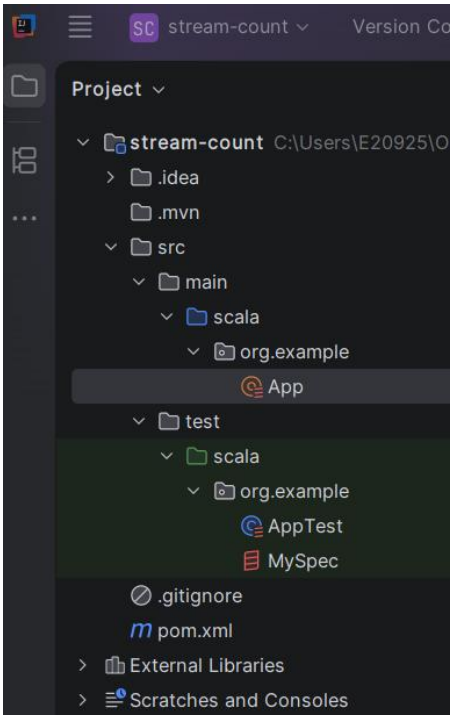


Click **Create**, you will see project **stream-count**

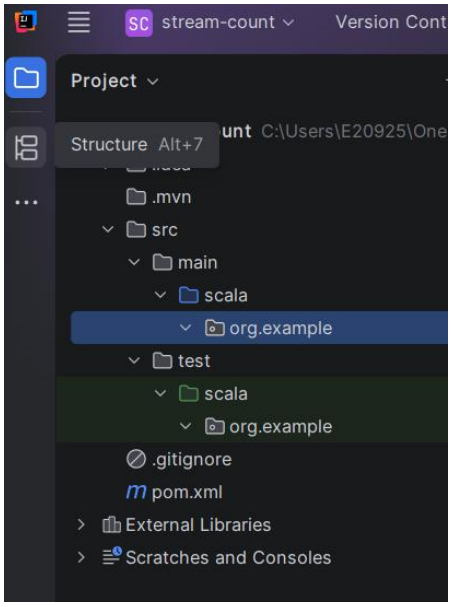


delete

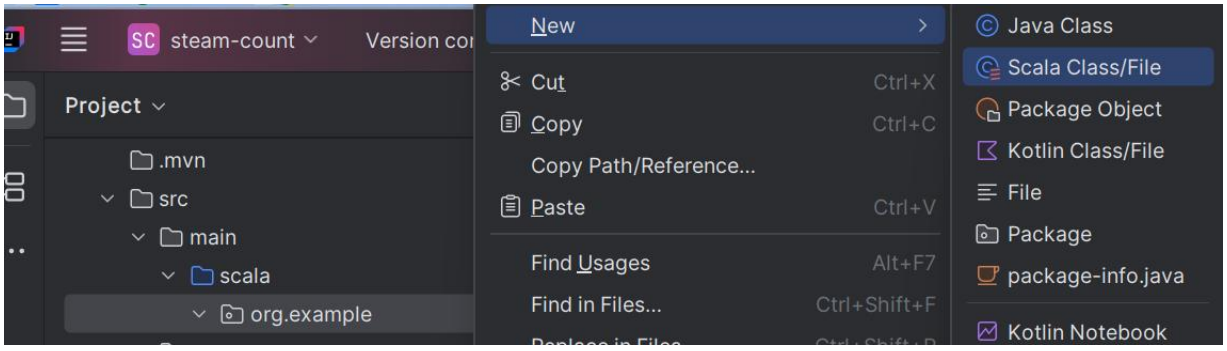
- **main.scala.org.example.App**
- **test.scala.org.example.App**
- **main.scala.org.example.MySpec**



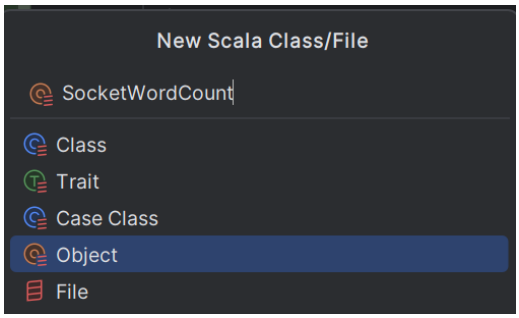




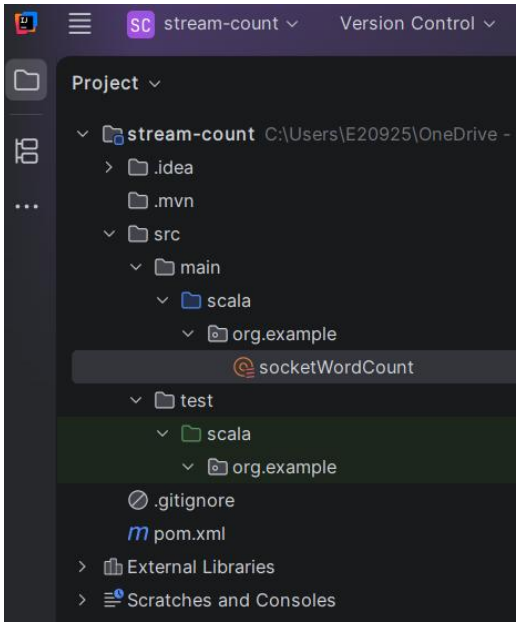
Create a new **scala** class under **main.scala.org.example** package



Name the new Scala Object "**SocketWordCount**", press **enter** key on keyboard.



A new file "**SocketWordCount.scala**" will be created



Copy and paste the code in **SocketWordCount.scala**



double-click **pom.xml**, change

```
<properties>
  <scala.version>2.7.0</scala.version>
</properties>
```

To

```
<properties>
  <scala.version>2.12.18</scala.version>
</properties>
```

Also, add the following dependencies in **pom.xml**

```
<dependency>
	<groupId>org.scala-tools</groupId>
	<artifactId>maven-scala-plugin</artifactId>
	<version>2.12</version>
</dependency>

<dependency>
	<groupId>org.apache.maven.plugins</groupId>
	<artifactId>maven-eclipse-plugin</artifactId>
	<version>2.5.1</version>
</dependency>

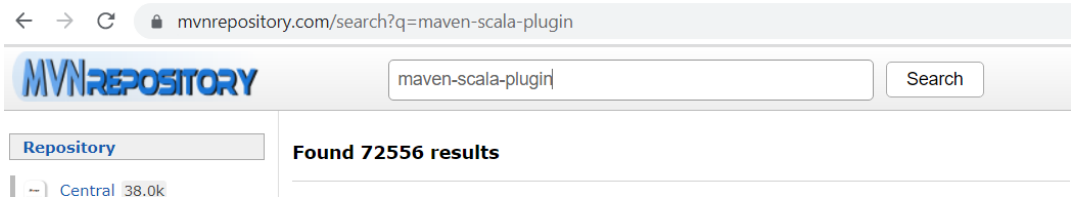
<!-- Source: https://mvnrepository.com/artifact/org.apache.spark/spark-core -->
<dependency>
	<groupId>org.apache.spark</groupId>
	<artifactId>spark-core_2.12</artifactId>
	<version>3.2.2</version>
	<!--<scope>compile</scope>-->
</dependency>

<!-- Source: https://mvnrepository.com/artifact/org.apache.spark/spark-streaming -->
<dependency>
	<groupId>org.apache.spark</groupId>
	<artifactId>spark-streaming_2.12</artifactId>
	<version>3.1.3</version>
	<!--<scope>provided</scope>-->
</dependency>

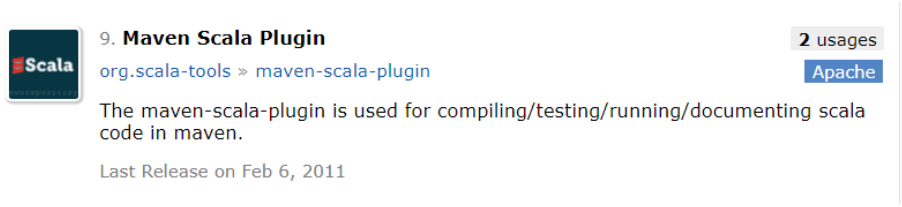
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core -->
<dependency>
	<groupId>org.apache.logging.log4j</groupId>
	<artifactId>log4j-core</artifactId>
	<version>2.24.3</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-collections/commons-collections -->
<dependency>
	<groupId>commons-collections</groupId>
	<artifactId>commons-collections</artifactId>
	<version>3.2.2</version>
</dependency>
```

The dependency can be extracted from https://mvnrepository.com/.
For “**maven-scala-plugin**”, for example, search for it in the site.



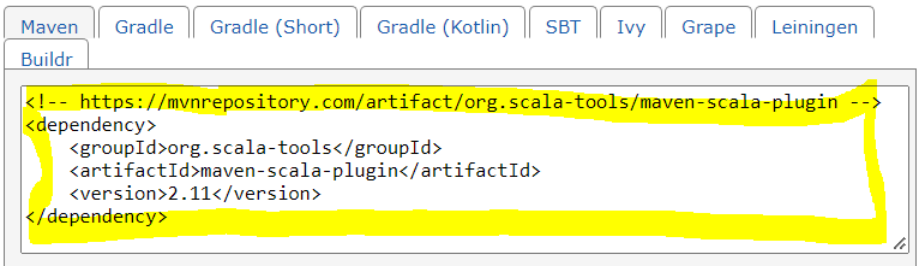
It will return:



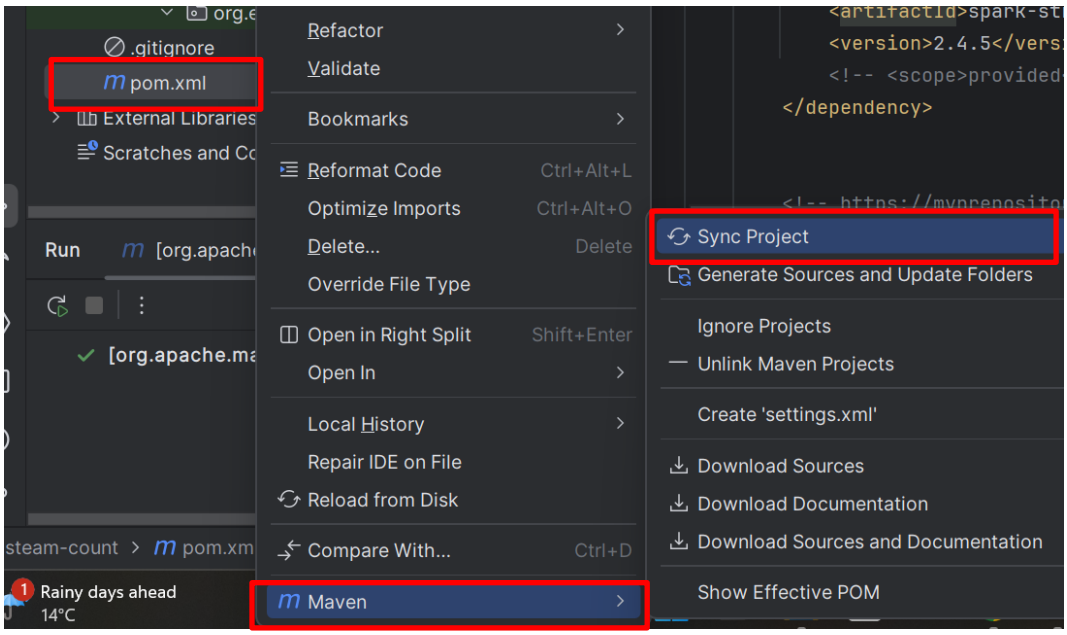
Click “**maven-scala-plugin**”, you will see a list where 2.11 is what we need.



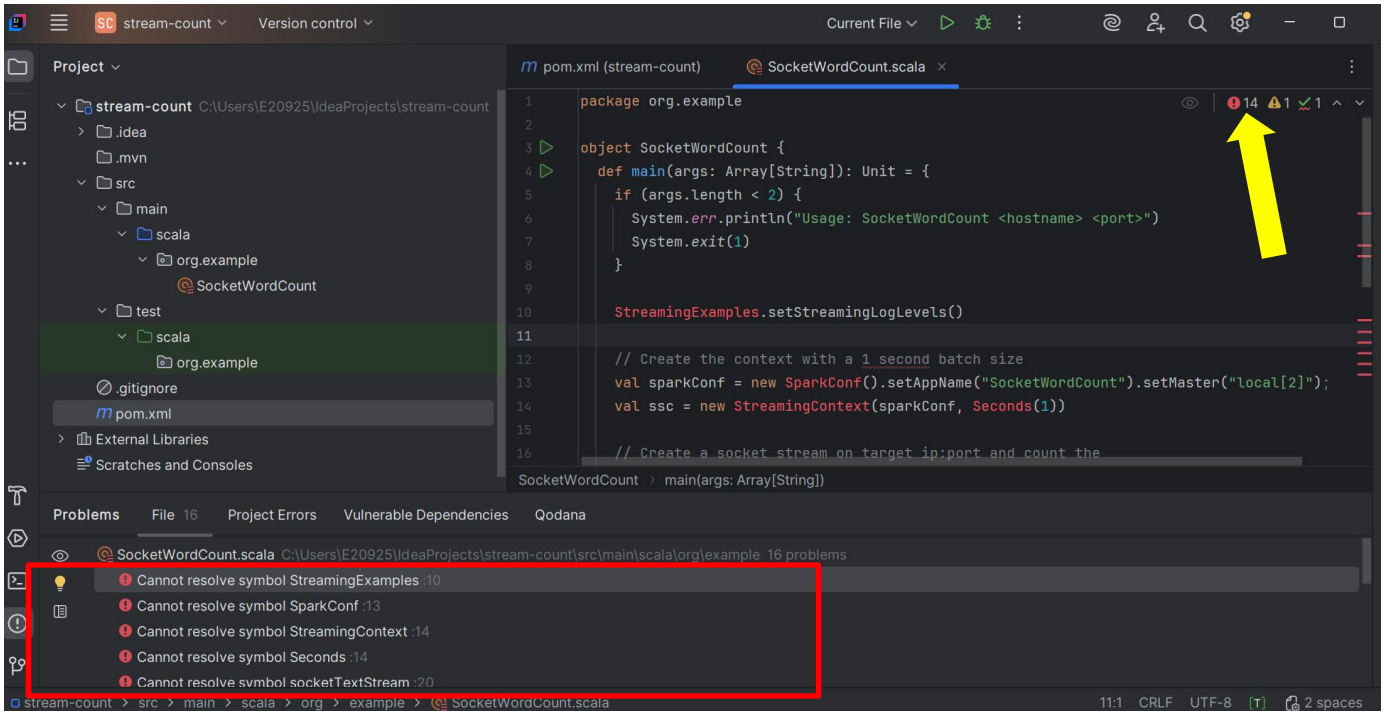
Click 2.11, you will see the following, copy and paste it to **pom.xml**



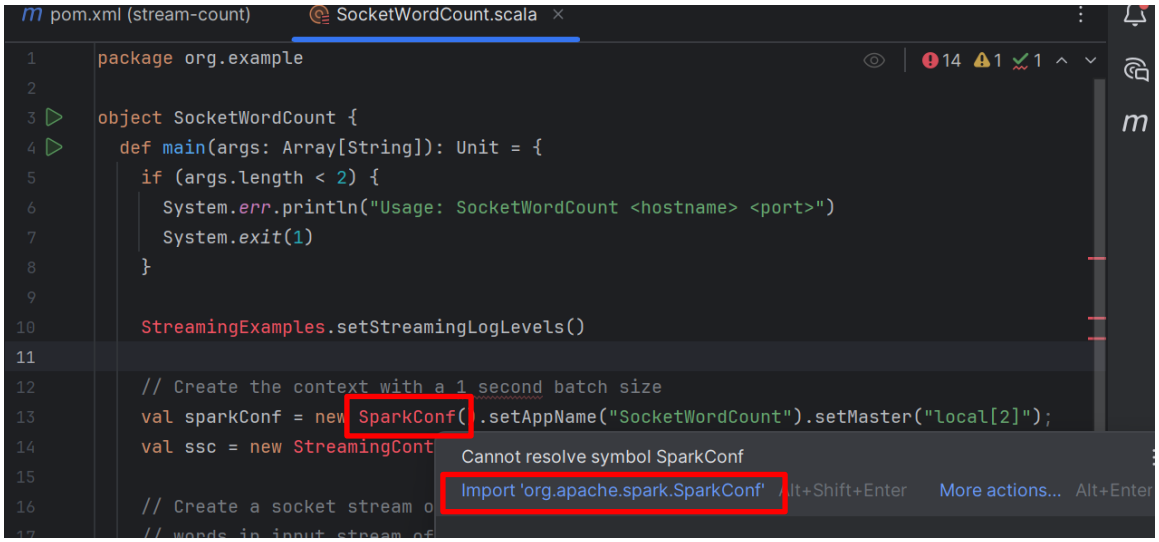
Right click **pom.xml,** select **Maven,** select **Sync Project**



Then, we fix the errors (highlighted by red) in "**SocketWordCount.scala**" (if errors are not highlighted red, you can try "File -> Invalidate Caches -> Invalidate and Restart" to fix)



To fix the error "**Cannot resolve symbol SparkConf**", stop cursor on **SparkConf**, you will see the following:

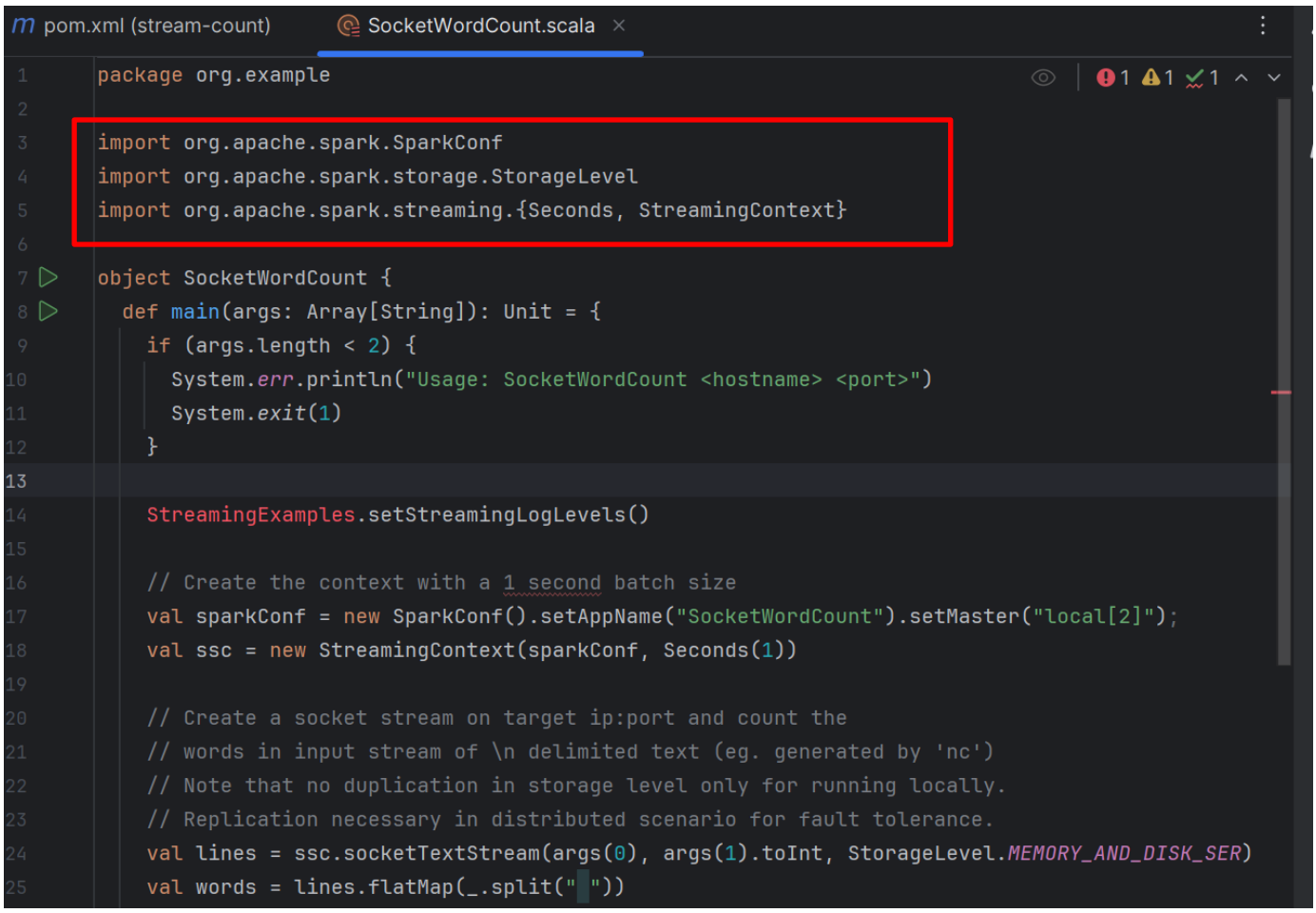


Click "**import 'org.apache.spark.SparkConf'**".

A new line "**import org.apache.spark.SparkConf**" will be added and the **SparkConf** is not highlighted red now.



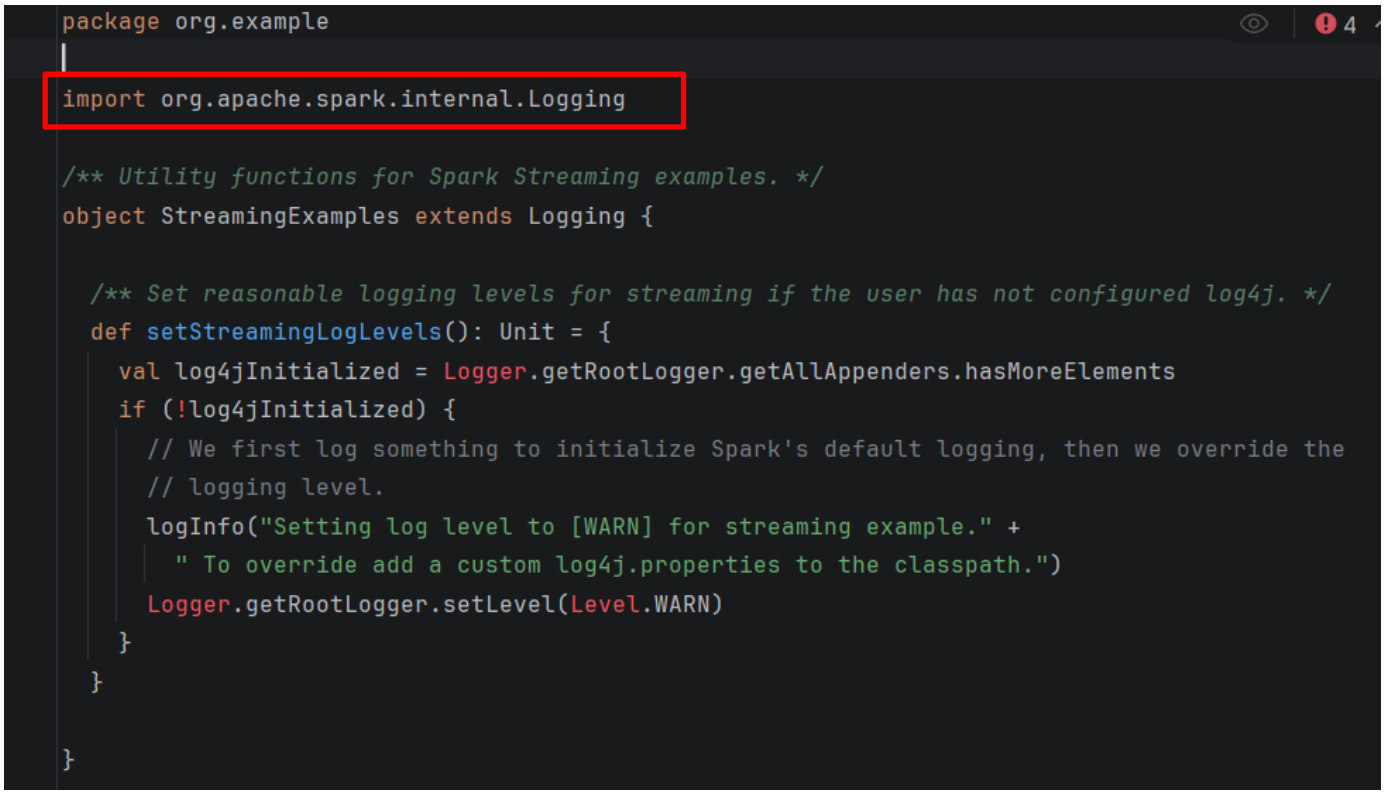
In a similar way, more packages are imported, and most errors can be fixed but StreamingExamples.



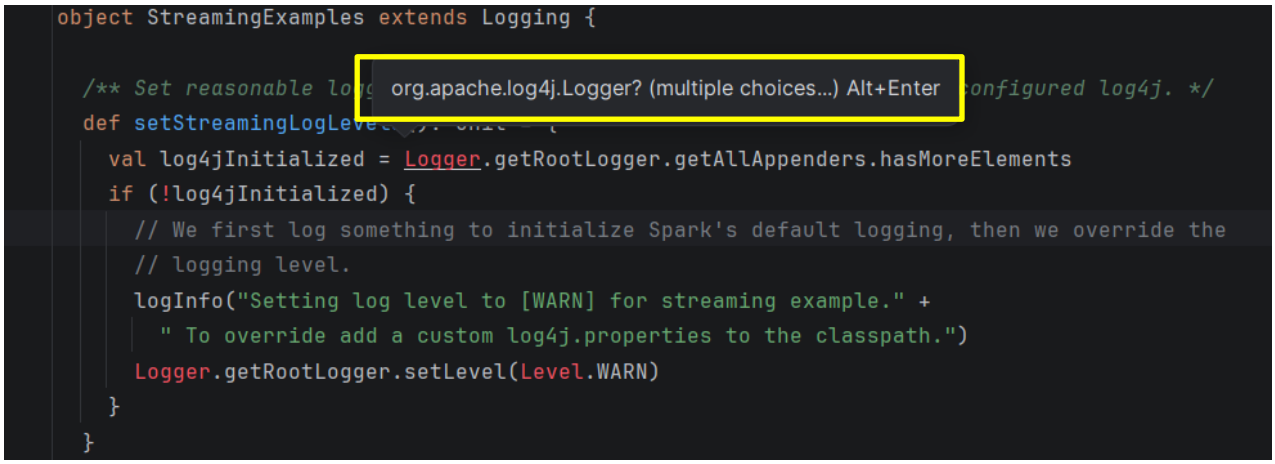
To fix this error "**cannot resolve symbol StreamingExamples**", we need add a new Scala object named "**StreamingExamples.scala**" under **main.scala.org.example** package in the same way as adding "**SocketWordCount.scala**". Then, you should see that all errors in "**SocketWordCount.scala**" are fixed.



Stop cursor on highlighted red **Logging** to import necessary package. The error should be fixed.



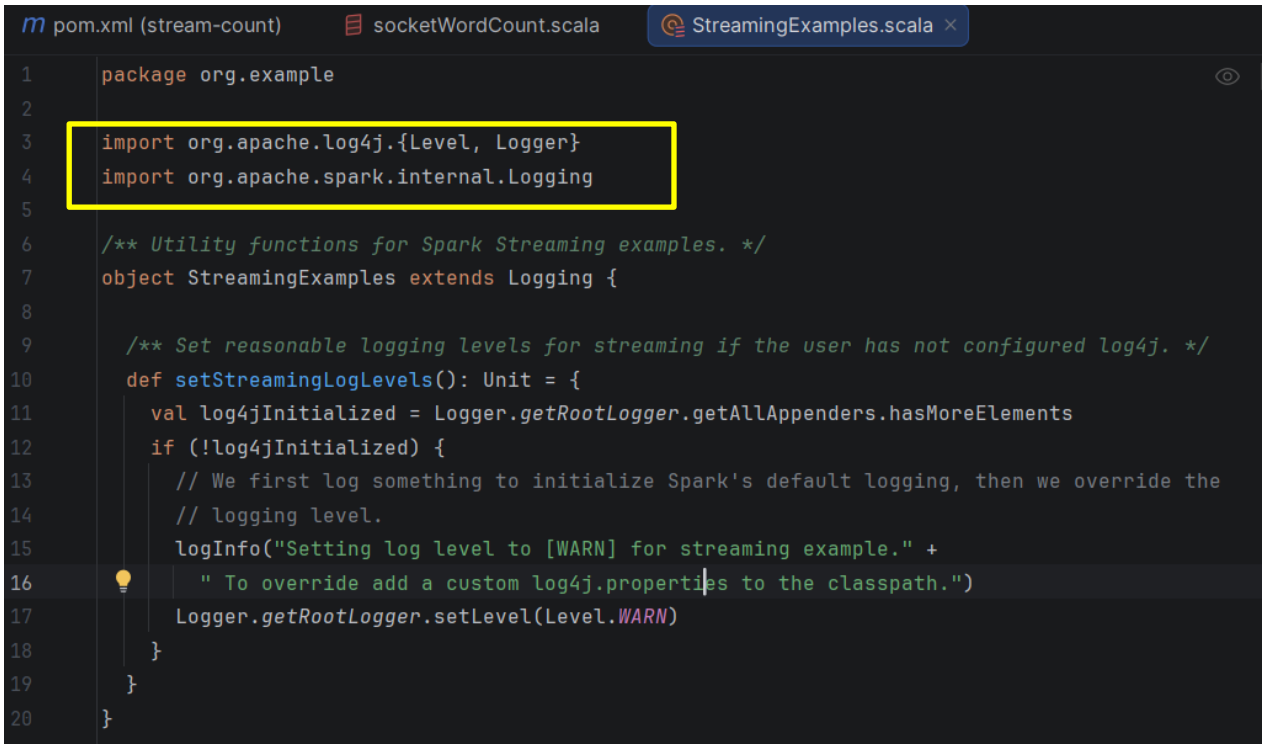
For highlighted red Logger, clicking **Logger**, you will see "… Alt+Enter". Do it, you will see a selection list and select **"org.apache.log4j.Logger"**. Similar, you repeat the same for **Level**.




```
object StreamingExamples extends Logging {

  /** Set reasonable lo                                    onfigured log4j. */
  def setStreamingLogLeve
    val log4jInitialized = Logger.getRootLogger.getAllAppenders.hasMoreElements
    if (!log4jInitialized) {
      // We first log something to initialize Spark's default logging, then we override the
      // logging level.
      logInfo("Setting log level to [WARN] for streaming example." +
        " To override add a custom log4j.properties to the classpath.")
      Logger.getRootLogger.setLevel(Level.WARN)
    }
  }
```

All errors have been fixed.

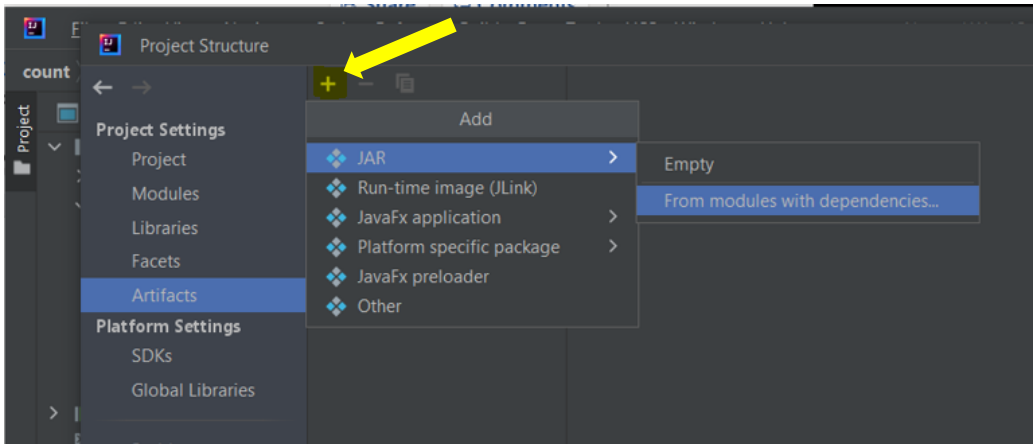


```
pom.xml (stream-count)   socketWordCount.scala   StreamingExamples.scala

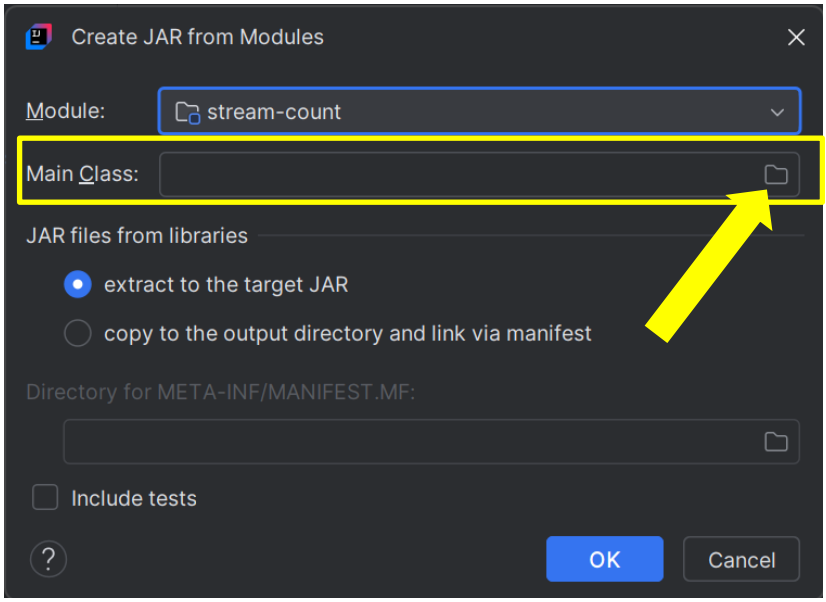
1   package org.example
2
3   import org.apache.log4j.{Level, Logger}
4   import org.apache.spark.internal.Logging
5
6   /** Utility functions for Spark Streaming examples. */
7   object StreamingExamples extends Logging {
8
9     /** Set reasonable logging levels for streaming if the user has not configured log4j. */
10    def setStreamingLogLevels(): Unit = {
11      val log4jInitialized = Logger.getRootLogger.getAllAppenders.hasMoreElements
12      if (!log4jInitialized) {
13        // We first log something to initialize Spark's default logging, then we override the
14        // logging level.
15        logInfo("Setting log level to [WARN] for streaming example." +
16          " To override add a custom log4j.properties to the classpath.")
17        Logger.getRootLogger.setLevel(Level.WARN)
18      }
19    }
20  }
```

Now, create an executable .jar file (the jar file is deployable, i.e., it includes all dependencies and thus its size is large)

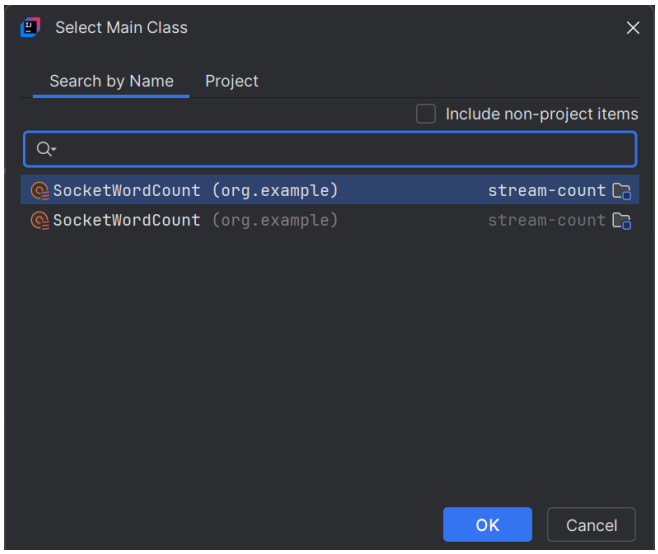
1. From the **File** menu, select **Project Structure...**.
2. From the **Project Structure** window, navigate to **Artifacts** > **the plus symbol + > JAR** > **From modules with dependencies...**.



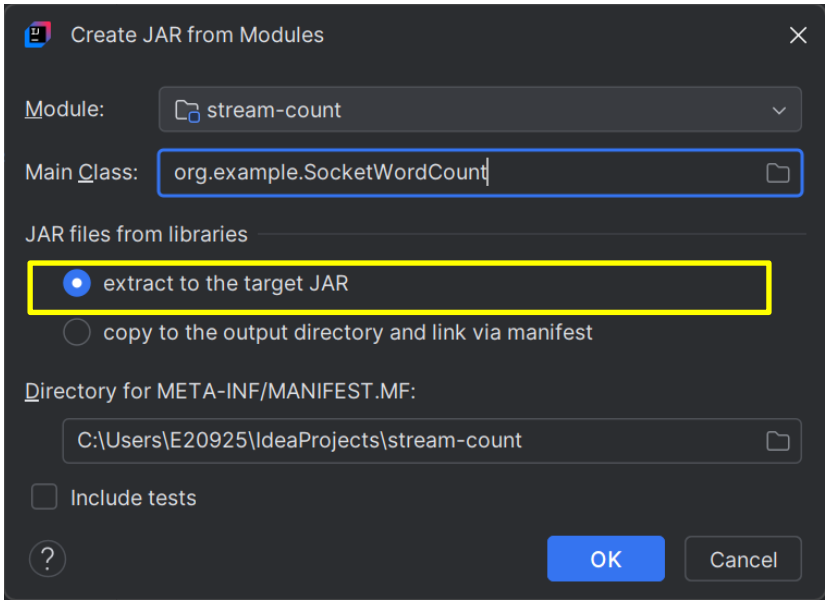
3. In the **Create JAR from Modules** window, select the folder icon in the **Main Class** text box.



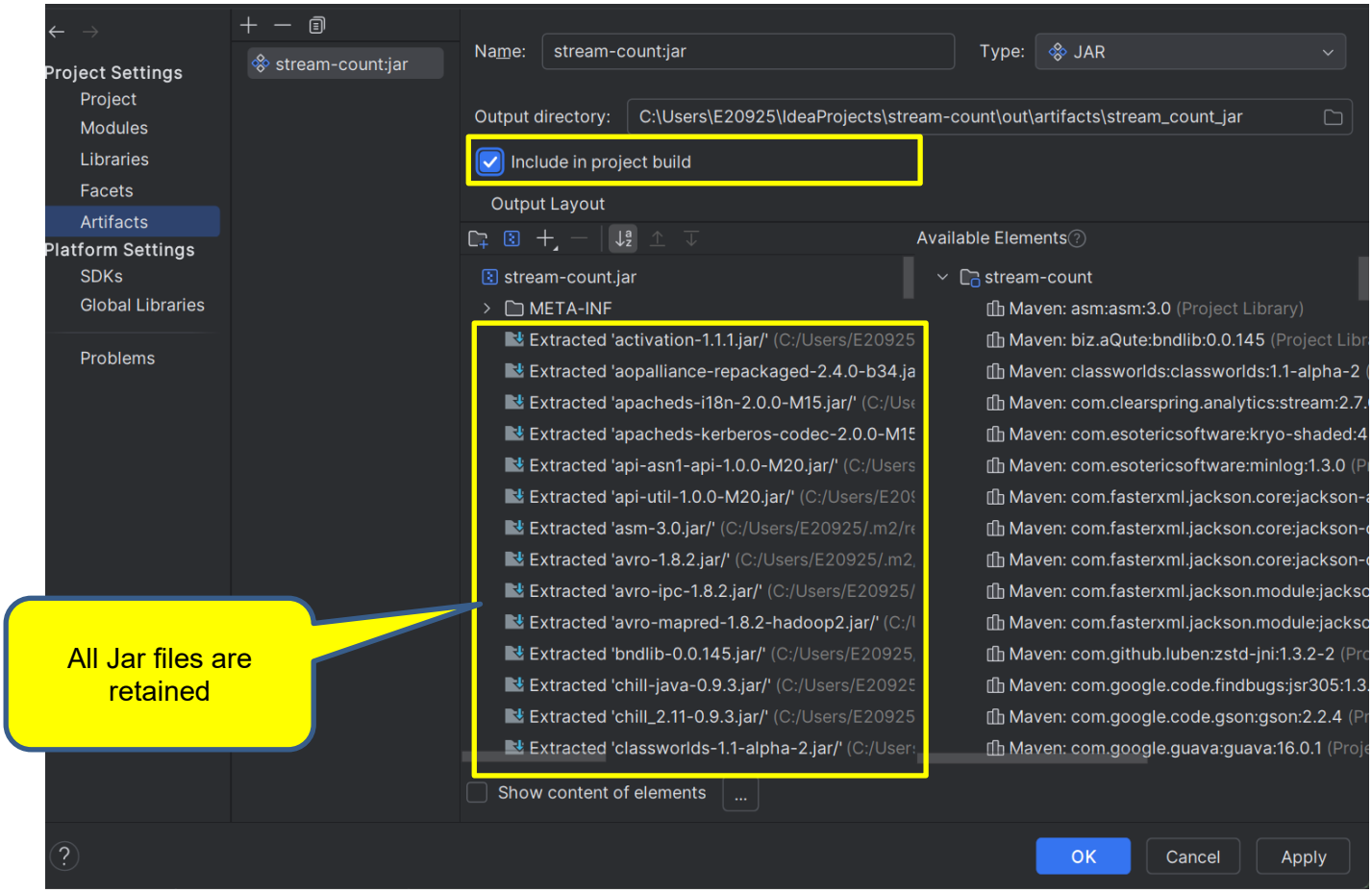
4. In the **Select Main Class** window, select the class that appears by default and then select **OK**.



5. In the **Create JAR from** Modules window, ensure the **extract to the target JAR** option is selected, and then select **OK**. This setting creates a single JAR with all dependencies.



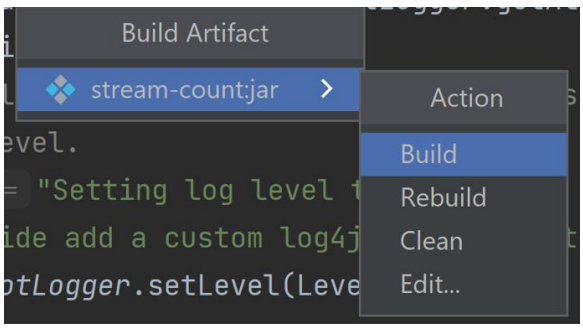
6. The **Output Layout** tab lists all the jars that are included as part of the Maven project. Note we want to build an executable jar. So, all jars are retained.



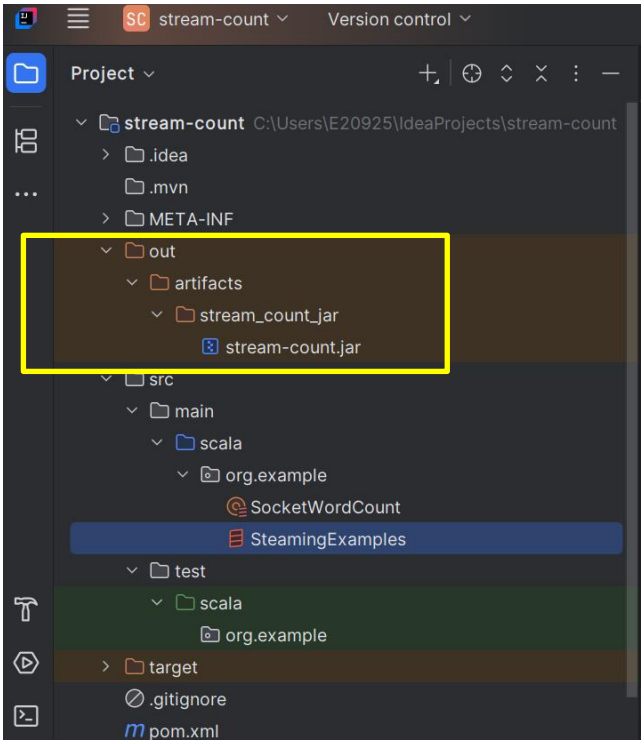
Tick the **Include in project build** checkbox. This option ensures that the jar is created every time the project is built or updated.

Select **Apply** and then **OK**.

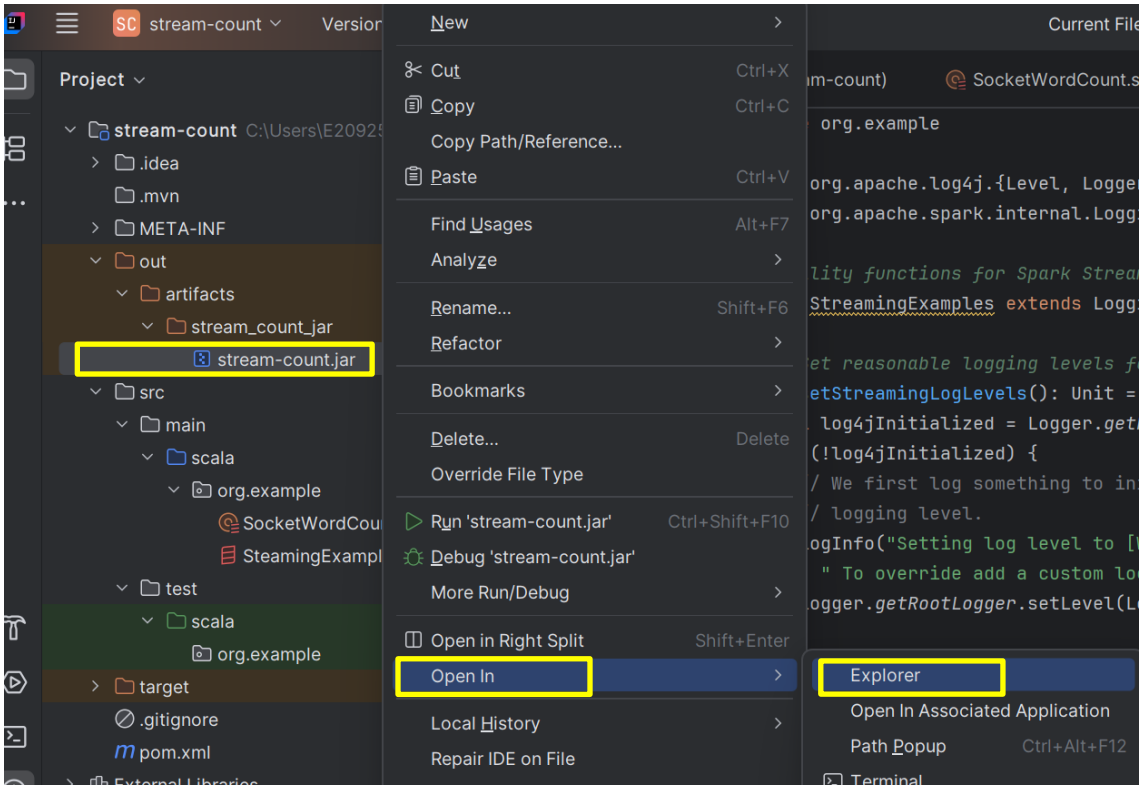
7. **Build -> Build Artifacts…**
You will see the following and click **Build**



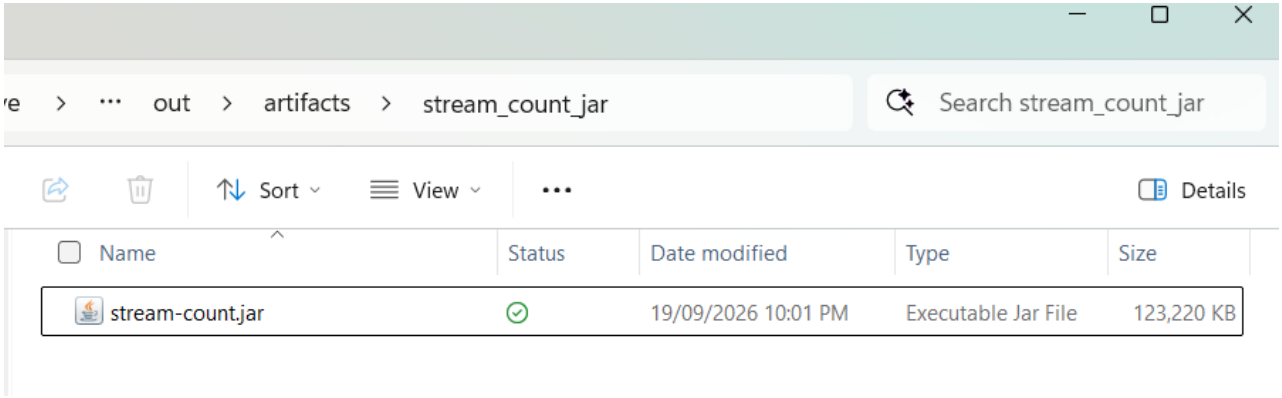
If running successfully, **stream-count.jar** is created in a new directory **out**



You can go to the folder where the executable jar is:



The Jar file size is large since all dependencies are included.

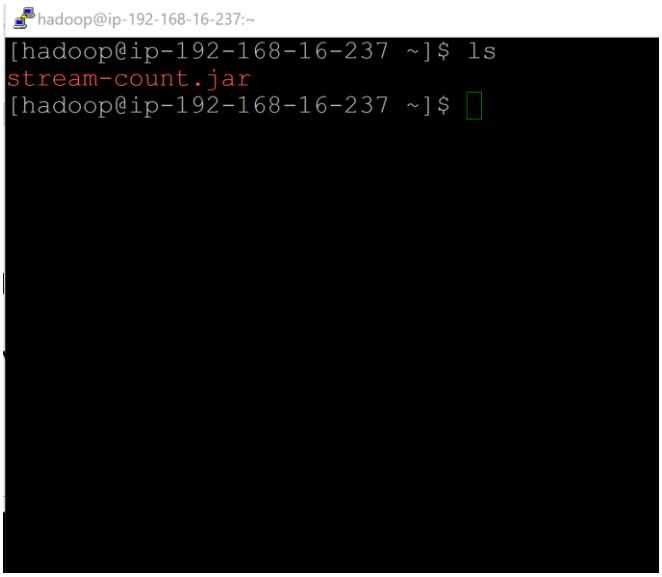


Upload executable JAR file to AWS EMR cluster master node and Run it.

1. Upload **stream-count.jar** to AWS EMR cluster master node.

   a) **Open two terminals**

First terminal:

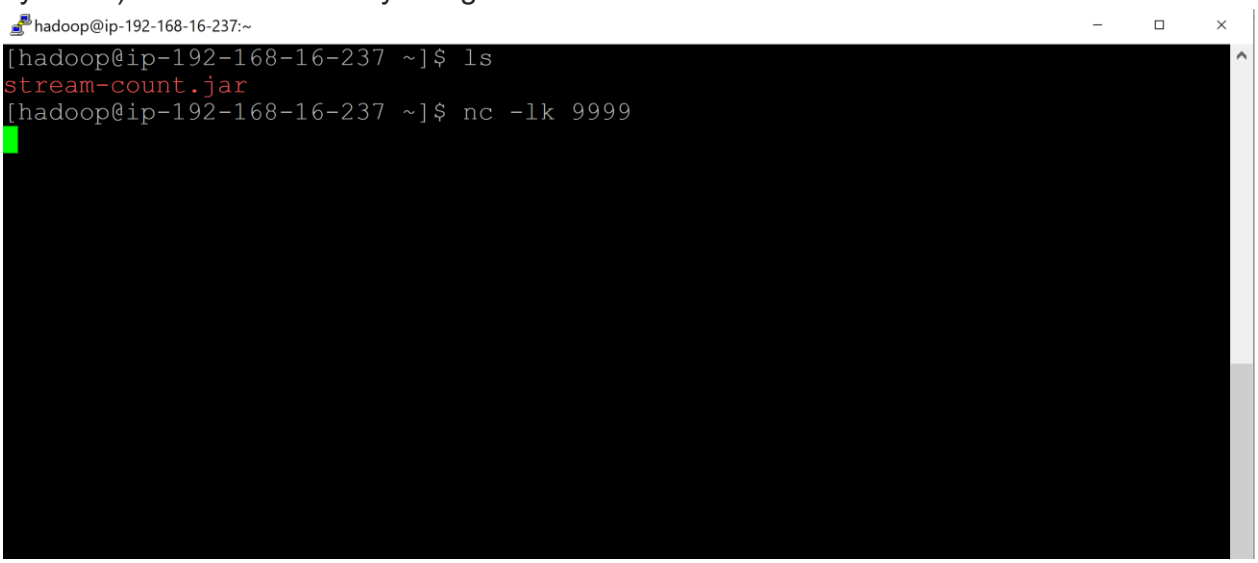


Second Terminal:



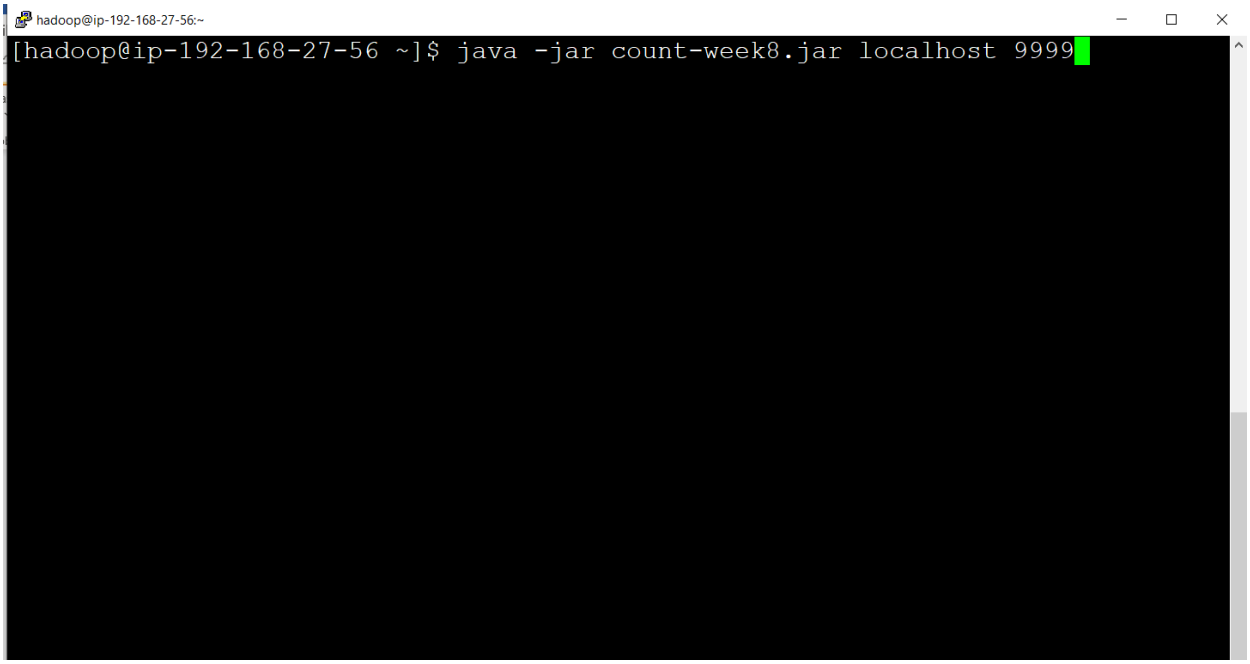
b) Run a data server in the first terminal and jar file in the second terminal.

First terminal: You will first need to run Netcat (a small utility found in most Unix-like systems) as a data server by using "**nc -lk 9999**".



Second terminal: run the jar file

$ java -jar stream-count.jar localhost 9999

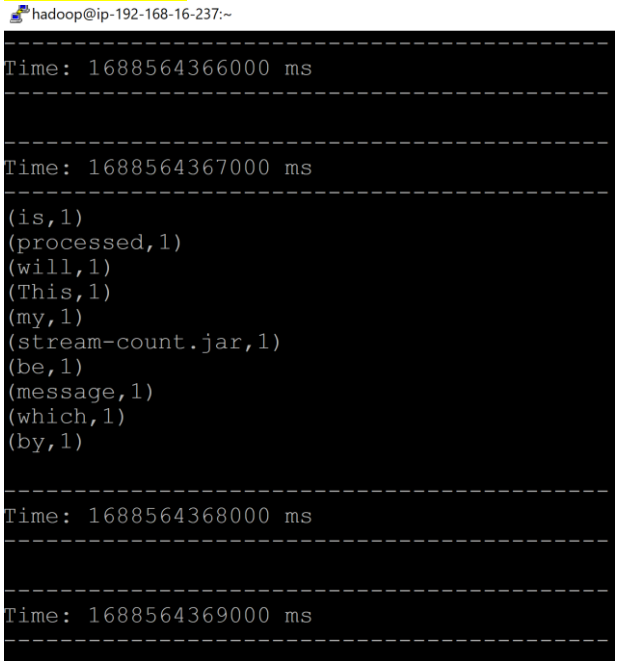


c) Input any text in the first terminal and observe what happened in the second terminal.

First terminal, input "This is my message which will be processed by stream-count.jar"

```
[hadoop@ip-192-168-16-237 ~]$ nc -lk 9999
This is my message which will be processed by stream-count.jar
```

Second terminal: After inputting message in the first terminal, the second terminal will print word count results.



Summary:

The message input in the first terminal is a streaming data at port 9999 which is received by the SocketWordCount class monitoring port 9999. The text message is then processed and print out the word count.

- Tips: SparkSession vs SparkContext

A SparkSession is the entry point to using Spark functionality. It manages the configuration of the Spark application and coordinates the creation of RDDs, DataFrames, and SQL tables, among other things. A SparkContext, on the other hand, represents the connection to a Spark cluster and is used to configure the cluster resources, execute tasks, and collect results.

In most cases, a SparkSession should be used as it includes the functionality of a SparkContext and more. However, in some advanced scenarios, it may be necessary to use a SparkContext directly.

Don't forget to terminate your cluster at end of the lab class!

*Also, if trying out any these exercises outside of the lab class, make sure you terminate your cluster and **DO NOT** leave clusters running idle for days.*

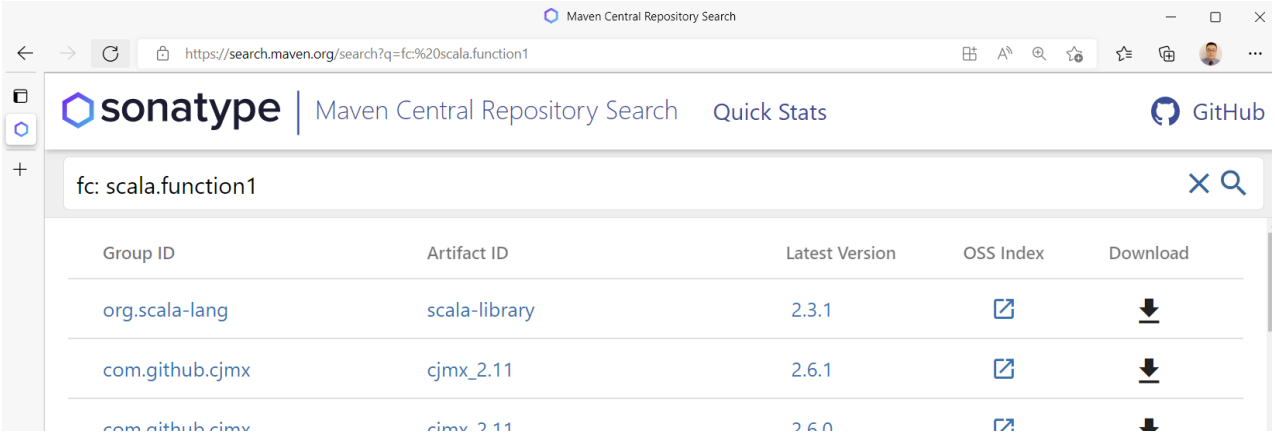
Useful Information

- **How to handle Exception in thread "main" java.lang.NoClassDefFoundError: xxx?**

Let's use **count-week8.jar** as an example. When executing **count-week8.jar,** the following error message may be returned. It means **count-week8.jar** missed some dependencies.

```
[hadoop@ip-192-168-27-56 ~]$ java -jar count-week8.jar localhost 9999
Exception in thread "main" java.lang.NoClassDefFoundError: scala/Function1
        at rmit.SocketWordCount.main(SocketWordCount.scala)
Caused by: java.lang.ClassNotFoundException: scala.Function1
        at java.net.URLClassLoader.findClass(URLClassLoader.java:387)
        at java.lang.ClassLoader.loadClass(ClassLoader.java:418)
        at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:352)
        at java.lang.ClassLoader.loadClass(ClassLoader.java:351)
        ... 1 more
```

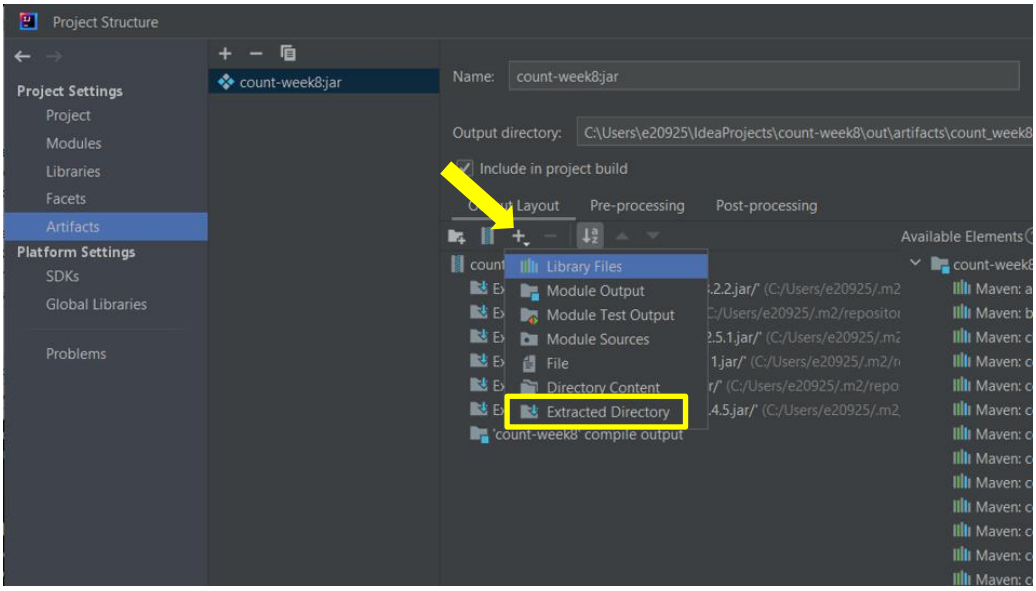
1. Searching **Artifact ID** related to class **scala/function1** using format as "**fc:scala.function1**" in Maven Central (https://search.maven.org/?eh=). From the search results, we noticed **scala-library** should be included in **count-week8.jar** but it is missing yet.



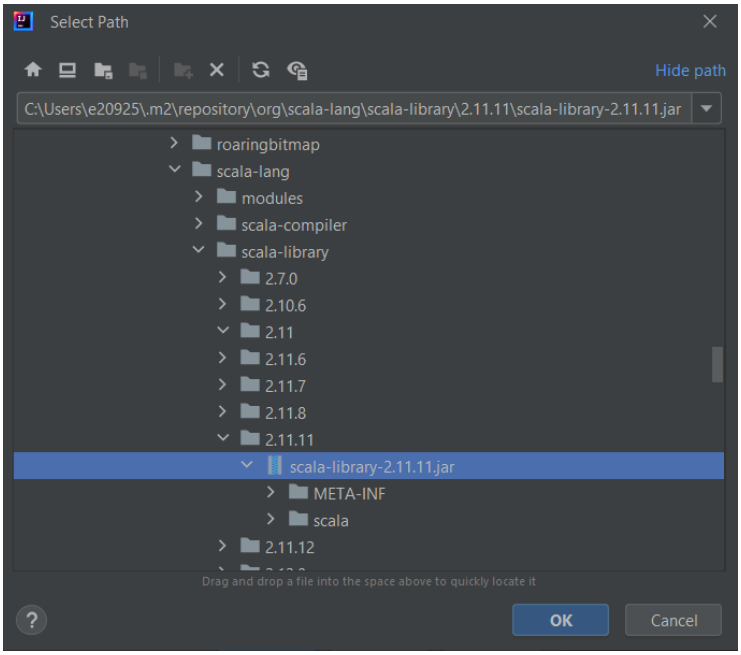
2. Check whether **org.scala-lang:scala-library** has been included in pom.xml as a dependency. If not, go to **Maven Repository** online to find the relevant repository and add in pom.xml, do not forget to right click pom.xml, **Maven->Reload Project**.

```
<dependencies>
  <dependency>
    <groupId>org.scala-lang</groupId>
    <artifactId>scala-library</artifactId>
    <version>${scala.version}</version>
  </dependency>
```

3. To add the missing jar file, go back to (Step 6) **File -> Project Structure… -> Artifacts**, click "+" and select **Extracted Directory**



Select **C:\Users\e20925\.m2\repository\org\scala-lang\scala-library\2.11.11\scala-library-2.11.11.jar**



Click **OK.** Scala-library-2.11.11.jar will be included **Output Layout** of **count-week8.jar**. Click **Apply** and click **OK**

4. Build the count-week8.jar again by selecting **Build->Build Artifacts…** and then clicking **Build.**

Additional Knowledge of Spark programming (selective)

If you are interested, you can follow the introductions below to learn how to use the broadcast and accumulator of Spark by yourself. It is not necessary for this course.

- ## Broadcast

In Spark RDD, Broadcast variables are read-only shared variables that are cached and available on all nodes in a cluster in order to access or be used by the tasks. Instead of sending this data along with every task, spark distributes broadcast variables to the machine using efficient broadcast algorithms to reduce communication costs.

> For example, assume you are getting a two-letter country state code in a file and you want to transform it to a full state name, (for example CA to California, NY to New York etc.) by doing a lookup to reference mapping. In some instances, this data could be large and you may have many such lookups (like zip code). Instead of distributing this information along with each task over the network (overhead and time-consuming), we can use the broadcast variable to cache this lookup info on each machine, and tasks use this cached info while executing the transformations.

A sample code of broadcast is given in the weekly module with the name **RDDBroadcast.scala**.

```
package org.example
import org.apache.spark.sql.SparkSession

object RDDBroadcast extends App {
  val spark = SparkSession.builder()
    .appName("SparkByExamples.com")
    .master("local")
    .getOrCreate()

  val states = Map(("NY","New York"),("CA","California"),("FL","Florida"))
  val countries = Map(("USA","United States of America"),("IN","India"))
  val broadcastStates = spark.sparkContext.broadcast(states)
  val broadcastCountries = spark.sparkContext.broadcast(countries)

  val data = Seq(("James","Smith","USA","CA"),
    ("Michael","Rose","USA","NY"),
    ("Robert","Williams","USA","CA"),
    ("Maria","Jones","USA","FL")
  )
  val rdd = spark.sparkContext.parallelize(data)
  val rdd2 = rdd.map(f=>{
    val country = f._3
    val state = f._4
    val fullCountry = broadcastCountries.value.get(country).get
    val fullState = broadcastStates.value.get(state).get
    (f._1,f._2,fullCountry,fullState)
  })
  System.out.println(rdd.collect().mkString("\n"))
  System.out.println(rdd2.collect().mkString("\n"))
}
```

You can create a new project to test **RDDBroadcast.scala** by following the steps for project **SparkWordCount** which is detailed in the last week. The only difference is you need to add one more dependency in the pom.xml

```
<!-- https://mvnrepository.com/artifact/org.apache.spark/spark-sql -->
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.12</artifactId>
  <version>3.0.3</version>
  <scope>provided</scope>
</dependency>
```

Once the jar file of the project is built, say **RDDBroadcast.jar**, upload it to the AWS EMR master node and run it

```
$ spark-submit --class org.example.RDDBroadcast --master yarn --deploy-mode client RDDBroadcast.jar
```

```
[hadoop@ip-192-168-23-43 ~]$ spark-submit --class org.example.RDDBroadcast --mas
ter yarn --deploy-mode client RDDBroadcast.jar
```

If run successfully, the output looks like:

```
(James,Smith,USA,CA)
(Michael,Rose,USA,NY)
(Robert,Williams,USA,CA)
(Maria,Jones,USA,FL)
```

```
(James,Smith,United States of America,California)
(Michael,Rose,United States of America,New York)
(Robert,Williams,United States of America,California)
(Maria,Jones,United States of America,Florida)
```

- **Accumulator**

Spark Accumulators are shared variables that are only “added” through an associative and commutative operation and are used to perform counters (Similar to Map-reduce counters) or sum operations.

A sample code of broadcast is given in the weekly module with the name **RDDAccumulator.scala**.

```scala
package org.example

import org.apache.spark.sql.SparkSession

object RDDBroadcast extends App {
    val spark = SparkSession.builder()
    .appName("Accumulator")
    .master("local")
    .getOrCreate()

    //example 1
    val threshold = args(1).toInt
    var blankline = spark.sparkContext.longAccumulator("Blank Lines")
    sc.textFile(args(0)).foreach { line =>
        if (line.length() > threshold) blankline.add(1)
    }
    System.out.println(blankline)

    //example 2
    val longAcc = spark.sparkContext.longAccumulator("SumAccumulator")
    val rdd = spark.sparkContext.parallelize(Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
    rdd.foreach(x => longAcc.add(x))
    System.out.println(longAcc.value)
}
```

You can create a new project to test **RDDAccumulator.scala** by following the same steps for project **RDDBroadcast** detailed above. Once the jar file of the project is built, say **RDDBroadcast.jar**, upload it to the AWS EMR master node and run it.

```
$ spark-submit --class org.example.RDDAccumulator --master yarn --deploy-mode client
RDDAccumulator.jar hdfs:///RDDAccum.txt 8
```

```
[hadoop@ip-192-168-23-43 ~]$ spark-submit --class org.example.RDDAccumulator --m
aster yarn --deploy-mode client RDDAccumulator.jar hdfs:///RDDAccum.txt 8
```

If run successfully, the output looks like:

```
The value returned by accumulator is 180
**********The value returned by accumulator

The value returned by accumulator is 55
```